MSSE 601

SEF 3

10/18/26

Jonathan Dereje, Abhinav Koukuntla

Week 8 - Project Component: Completed Software Project Plan & Presentation

1. Introduction and Approval Case

Golden Bike Shop is a four-location bicycle retailer, rental business, and repair shop. It sells bicycles, parts, and accessories online and in stores; rents approximately 400 bikes, including e-bikes; and repairs both rental and customer-owned bikes. The proposed system is a centralized web application for customers, store employees, and store managers.

The system will provide one reliable view of products, inventory, rental bikes, reservations, repairs, service bays, online orders, and store pickup. Its primary business value is preventing the conflicts that occur when those activities are managed separately: a bike being offered while under repair, a product being oversold, a reservation being double-booked, or a service bay being scheduled for two repairs at once.

The team recommends approval of a modular n-tier monolith with one authoritative relational database and asynchronous workers for non-blocking work. This design favors consistency and simpler cross-store transactions over the independent deployment and operational complexity of microservices. The first implementation should be a vertical slice through authentication, shared data, inventory search, rental reservation approval, and repair status because those capabilities establish the controls that the remaining features depend on.

2. Project Charter and Process Selection

2.1 Business problem and stakeholders

The current-process assumption is that the four locations do not have one reliable, shared view of inventory, rental availability, repairs, and service scheduling. That creates inaccurate availability, scheduling conflicts, missed sales and rentals, and additional risk during seasonal demand or local events.

The stakeholders are:

Stakeholder	Interest in the system
Shop owner	Financial visibility, business performance, costs, and approval of the investment
Store manager	Cross-store oversight, approvals, inventory, staffing, rentals, repairs, and service capacity
Store employee	Day-to-day sales, inventory, rental, repair, and pickup workflows
Customer	Product purchasing, bike reservations, repair requests, order status, and pickup information

2.2 Scope

The initial system includes:

- Product inventory views across all four stores
- Rental-bike availability, pricing options, and reservations
- Employee management of reservations
- Manager review and approval of pending reservations
- Repair tickets and repair-status updates
- Service-bay availability and appointments
- Parts usage that deducts from store inventory
- Online product orders with in-store pickup
- Customer access to their own orders, reservations, and repair information
- Role-based access for customers, store employees, and store managers
- Audit records for important business changes

The initial system excludes:

- Employee payroll and compensation
- Supplier and manufacturer management
- Delivery and shipping logistics
- A dedicated mobile application
- Payment processing design beyond recording the order transaction
- A separate employee-account data model; authentication and authorization are application concerns
- Telemetry or analytics entities not required by the submitted requirements

2.3 Assumptions, constraints, and risks

The plan assumes that Golden Bike Shop has suitable computers, network connectivity, a development budget, training capacity, stakeholder access, and usable source data for migration. The course schedule, team availability, stakeholder availability, and technical experience constrain the amount of functionality that can be designed and implemented.

The main risks are changing requirements, conflicting stakeholder priorities, inconsistent source data, cross-store transaction errors, seasonal traffic spikes, and insufficient time for testing. The selected architecture reduces distributed-data risk but concentrates operational risk in the application and shared database. The deployment and maintenance plan in Section 9 addresses that risk without changing the logical architecture.

2.4 Process selection

The team selected an iterative Agile process using Scrum practices. The weekly course components act as increments. The team maintains a shared backlog of requirements, open questions, reconciliation issues, and design decisions; reviews each increment against the Golden Bike Shop brief; and carries unresolved work into the next increment.

Scrum was selected over Waterfall because the requirements for inventory, rentals, repairs, and service bays were not fully defined at the start and were expected to change as the relationships between the

business areas became clearer. The tradeoff is less up-front predictability and a greater need for regular communication, review, and backlog maintenance.

3. Requirements Specification

3.1 User roles and access boundary

The system has three primary roles:

- **Customer:** May view the customer's own orders, reservations, and repair information and perform only the self-service operations explicitly allowed by the workflow.

- **Store employee:** May perform authorized day-to-day retail, inventory, rental, repair, service-bay, and pickup tasks.

- **Store manager:** May review approvals and view or manage information across all four stores according to authorization policy.

The role distinction is enforced in the Web/API layer for every protected operation and record. A role-specific screen is not treated as a security boundary by itself.

3.2 Functional requirements

ID	Requirement and acceptance summary	Priority
FR-01	An authorized store user can search products by name or product identifier and see quantity and availability by store. Employees use this for authorized daily operations; managers have cross-store visibility. The system displays a clear out-of-stock result when no store has the product.	3
FR-02	A customer can select a store, rental period, start and end times, and hourly, daily, or weekly pricing; search results contain only bikes available for the full period and exclude bikes that are reserved, rented, or under repair. The system performs a final availability check before recording a reservation request.	4
FR-03	An employee can create and update a repair ticket. When a bike is under repair, its status	5

	changes to unavailable and it is removed from rental search results. A completed repair must pass the completion workflow before the bike can become available again.	
FR-04	A customer can select available bicycles, parts, or accessories, specify quantity, and submit an online order. The system prevents an order from exceeding available inventory and records the products, quantities, status, and total amount.	9
FR-05	A customer can select an eligible pickup store. The system saves that store with the order, and an employee can move the order through ready-for-pickup and collected statuses.	10
FR-06	An authorized employee can locate, create, modify, and cancel a reservation. A modification repeats the availability check; cancellation changes status and releases the bike for later availability.	6
FR-07	An employee can view service-bay appointments by store, date, time, and bay and schedule an appointment only when it does not overlap another active appointment in the same bay.	8
FR-08	A customer can submit a repair request and view only that repair's status. An employee can schedule and update the repair. The initial	11

	employee repair workflow is the primary Week 4 journey; the customer request and tracking path remains in scope even though it was not selected as a separate journey.	
FR-09	An employee can record a product and quantity used on a repair ticket. The system deducts the quantity from the handling store's inventory and blocks the normal transaction when inventory is insufficient.	7
FR-10	A manager can review pending rental requests and approve one only when the selected bike is available for the entire period and is not rented or under repair. Approval changes the request to confirmed and prevents an overlapping active reservation.	12

3.3 Non-functional requirements

ID	Requirement and acceptance summary	Priority
NFR-01 Performance	Inventory, rental-availability, and service-bay searches return within three seconds for at least 95 percent of requests while supporting four stores, approximately 400 rental bikes, and up to 100 concurrent users.	13
NFR-02 Data consistency	Confirmed inventory, order, reservation, repair-status, and service-bay changes are visible to affected operations within ten seconds for at least 99 percent of updates when network connectivity is available. Errors or last-	1

	successful-update information must be shown instead of presenting stale data as current.	
NFR-03 Availability	Core inventory, order, reservation, repair, and service-bay functions are available during at least 99.5 percent of published operating hours each month, excluding scheduled maintenance announced at least 24 hours in advance.	14
NFR-04 Security	The system denies 100 percent of unauthorized access attempts in role-based security testing. Customers see only their own records; employees and managers receive only authorized operational or cross-store access.	2
NFR-05 Usability	After training, at least eight of ten employee or manager participants complete an assigned primary task within five minutes without assistance.	15

3.4 Prioritization rationale

The team prioritized data consistency first because every major function depends on accurate shared information. Security follows because the system contains customer and business records. Inventory is next because rental, ordering, and repair-parts workflows depend on it. Rental, repair, reservation management, parts usage, service bays, ordering, pickup, customer repairs, and approval then follow by dependency and business value. Performance, availability, and usability remain release obligations even though they appear later in the feature priority list.

4. User Journeys and Interface Design

4.1 Selected journeys

The three Week 4 journeys were retained because together they cover the highest-value cross-module activities:

1. **Reserve a rental bike:** Select period and location, search availability, select a bike, perform a final check, submit a pending request, and display the request status.

2. **Place an online product order for store pickup:** Search a product, choose quantity and pickup store, perform a final inventory check, create the order, prepare it, and mark it collected.
3. **Process a rental-bike repair:** Find the bike, create a repair ticket, mark it unavailable, schedule a bay, record parts, update status, inspect the result, and either release or retain the bike.

These journeys show the system's most important dependencies: availability rules use reservation and repair data; orders and repairs use store inventory; and service appointments use repair tickets and bay schedules.

4.2 Reconciled interface decisions

The integrated design uses the following interface behavior:

- **Rental reservation:** Use a calendar/date control, start and end time controls, store selection, rental option, and optional filters for bike type and price. Results show bike model, price, and pickup location. The screen does not promise a final confirmed reservation until the manager approval rule is satisfied.

- **Online order:** Use one product lookup field rather than three separate category search buttons. Results show description, price, available quantity, and store availability. The customer chooses quantity and pickup store before final submission. Category filtering is deferred because the current data model does not define a product-category field.

- **Rental-bike repair:** Search by bike identification number. After the bike is found, show current store, rental status, reservations, and repair history. The authenticated employee identity is read-only system context rather than a manually entered business field. The employee enters the problem description and submits the ticket.

- **Repair scheduling and status:** The employee workflow includes bay availability, appointment time, parts usage, status changes, and completion inspection. Customer repair requests and status views use the same repair-ticket records but expose only the requesting customer's records.

The screens use keyboard navigation, visible labels, readable text, spacing around controls, and text-based error messages. Color is not the only signal for status. The Week 7 usability plan must turn these design intentions into measured tasks and evidence.

5. Application Architecture

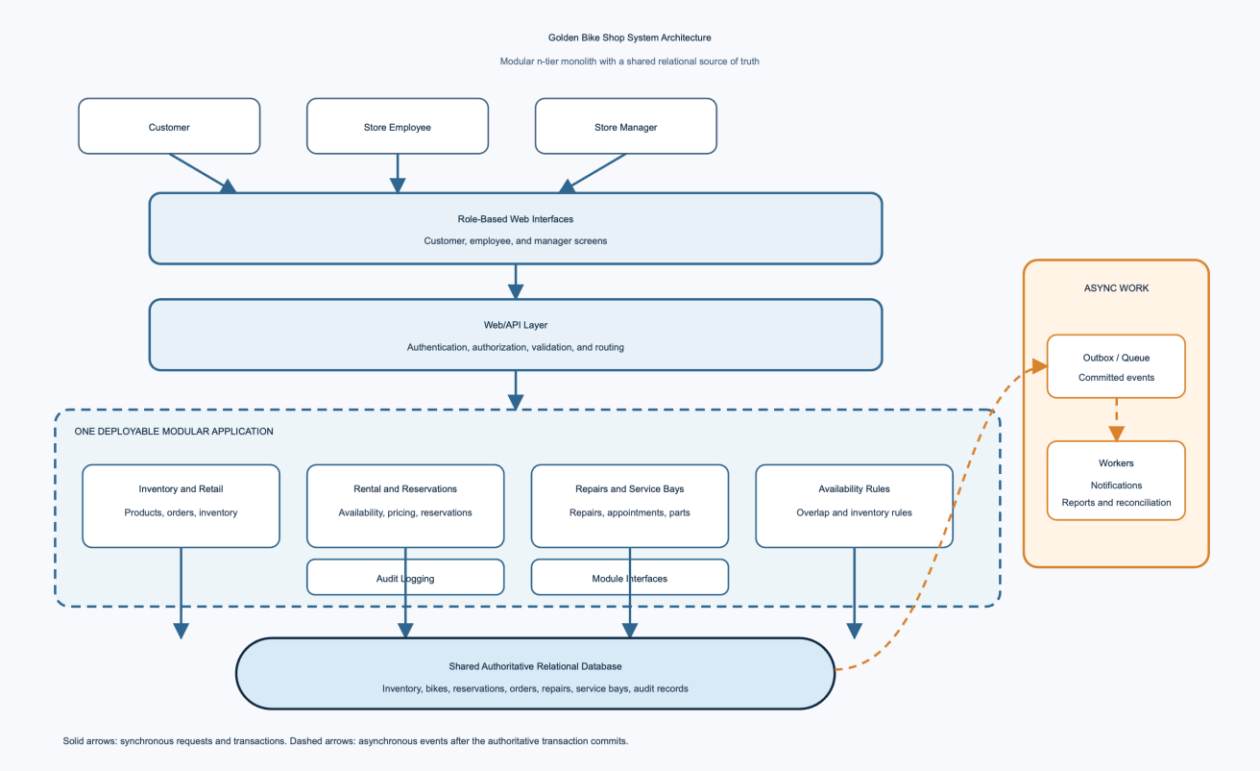


Figure 2. Recommended modular n-tier architecture.

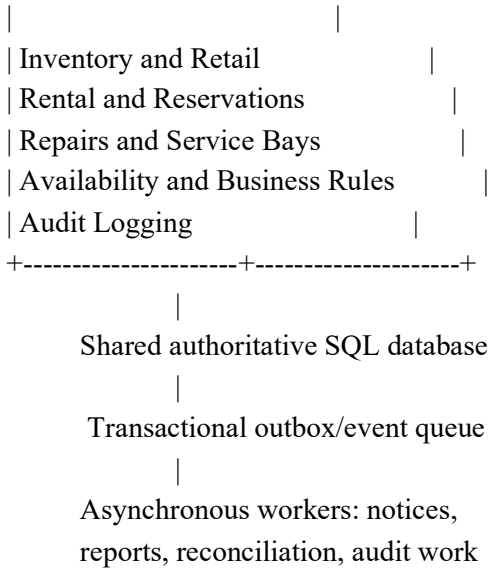
5.1 Architectural decision

The team selected a modular n-tier monolithic architecture with asynchronous workers. The role-based web interfaces form the presentation tier. A Web/API layer handles authentication, server-side authorization, validation, and routing. Business modules run within one deployable application and use one authoritative relational database. Workers handle notifications, reports, reconciliation, and other work that does not need to block the user's transaction.

The initial design deliberately does not use independently deployed microservices or separate databases for each store. The expected scale is four stores, approximately 400 rental bikes, and up to 100 concurrent users. At that scale, microservices would add network, deployment, monitoring, and distributed-transaction complexity without a demonstrated need for independent scaling.

5.2 Logical structure

```text
Customers, store employees, and store managers
                  |
          Role-based web interfaces
                  |
         Web/API: auth, authorization,
              validation, routing
                  |
    +---------------------------------------+
    | One deployable modular application    |
```



5.3 Module responsibilities

The application is divided into logical modules within one deployable application. This organization separates business responsibilities while allowing the modules to share a common database and participate in transactions when required.

- **Inventory and Retail:** Maintains product information and inventory quantities. Supports search of products, stock checks, order processing, order statuses, and inventory deductions for parts used in repair.
- **Rental and Reservations:** Searches for bikes available for rent, creates and modifies reservations, handles cancellations, and supports manager approvals.
- **Repairs and Service Bays:** Manages repair tickets, bike repair status, service-bay appointments, parts usage, and completion inspections.
- **Availability and Business Rules:** Applies shared rules for other modules such as inventory sufficiency, reservation conflicts, and appointment scheduling.
- **Audit Logging:** Records of important changes, including reservation approvals, inventory adjustments, repair status changes, and order status updates.

5.4 Transaction boundary

The shared relational database is the authoritative source of information across all four Golden Bike Shop store locations. Any operation that affects availability or inventory must use the database to prevent partially completed changes. For example, confirming a rental reservation requires checking that the bike is available for the requested period and recording the confirmation without allowing another concurrent request to reserve the same bike for an overlapping period. Similarly, placing an order requires checking available inventory and recording the order and its associated order lines as one consistent operation.

The application should use constraints and validation to protect against concurrent requests. An availability search alone is not enough because the underlying information can change before the user submits the request. Rental reservations, orders, repair parts, service bays, and bike repair statuses require extra attention to validation because it is important that multiple reservations are not able to be

confirmed for the same bike or service bay during conflicting periods and so that products are not accidentally oversold.

For background processing, the application can use the transactional outbox queue. The business transaction records an event in the database alongside the change. An asynchronous worker subsequently processes the event. Failed processing can be retried, and duplicate deliveries must not cause duplicate effects. The exact database engine and queue technology should be selected during implementation.

5.5 Quality-attribute support and tradeoffs

The selected architecture supports the system's goals in terms of data consistency, security, performance, availability and maintainability. A shared authoritative database and transactional business operations help keep inventory, reservations, orders, repairs, and service appointments consistent across all locations. Authentication and server-side role based authorization restricts access to customer records and operational functions. Customers may only access their own records while employees and managers can perform other authorized responsibilities. Database indexing, efficient queries, and asynchronous processing helps meet the requirements that at least 95% of inventory, rental availability, and service bay searches return within three seconds. Monitoring, database backups, controlled deployments, and documented recovery procedures support the target of 99.5% availability during published operating hours. Lastly, separating responsibilities into logical modules makes the code easier to understand, test, and maintain. The main tradeoff is that the application and database remain as shared operational dependencies. A defect, resource bottleneck, or deployment problem can affect multiple functions at once.

6. Data Design

6.1 Storage decision

The system uses a shared relational database for all four store locations. A relational database is a good choice here because the business depends on relationships among customers, products, inventory, rental bikes, reservations, repair tickets, service appointments, and orders. The shared database provides a consistent view of business information and supports transactions across related records. Store specific inventory and appointments remain associated with each respective store, while managers can access authorized information across each location.

The data design emphasizes controlled status changes and transactional updates. Primary keys uniquely identify records, foreign keys link relationships between tables, and constraints protect business rules. Access to sensitive customer information is controlled by the application's authorization rather than the database itself.

6.2 Canonical entities

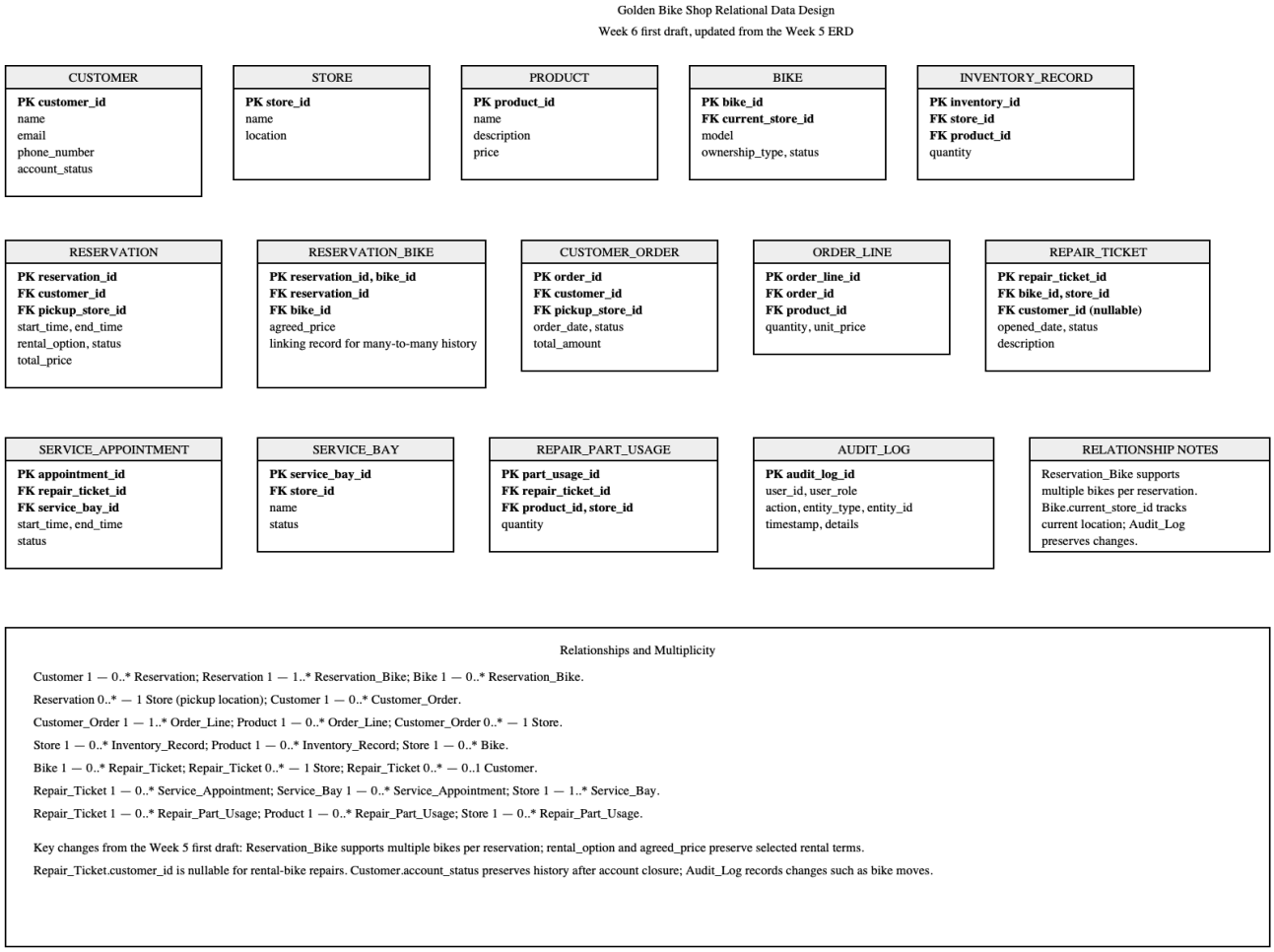


Figure 1. Revised Week 6 relational data design.

These entities provide the basis for inventory, rental, repair, scheduling, and online order workflows. Authentication and authorization are managed by the application rather than requiring a separate employee account entity in this initial data model.

- **CUSTOMER:** Stores customer contact information and account status
- **STORE:** Represents each individual Golden Bike Shop location
- **PRODUCT:** Stores product descriptions, names, and current prices.
- **BIKE:** Tracks each individual bike, current store, ownership type, model, and availability status
- **INVENTORY_RECORD:** Tracks the quantity of each product at each store
- **RESERVATION:** Stores customer rental requests, pickup store, rental dates, rental option, status, and total price
- **RESERVATION_BIKE:** Links reservations to individual bikes and preserves the agreed price per bike.
- **CUSTOMER_ORDER:** Stores customer orders, pickup location, date, status, and total amount.
- **ORDER_LINE:** Records the product, quantities, and unit prices.
- **REPAIR_TICKET:** Tracks a bike's repair work, handling store, opening date, status description, and optional customer.

- **SERVICE_APPOINTMENT:** Records a repair appointment's service bay, start and end times, and status.
- **SERVICE_BAY:** Represents a service bay belonging to a store
- **REPAIR_PART_USAGE:** Records the products, quantities, and store inventory associated with parts used in repairs.
- **AUDIT_LOG:** Records important changes, including user, action, affected identity, timestamp, and details.

6.3 Relationships and business constraints

The database design addresses important business edge cases and preserves historical information, such as multiple bikes per reservation. RESERVATION_BIKE supports a reservation containing several bikes and allows each bike to have multiple reservations over time. Rental prices may change during seasonal spikes; however, the data preserves the total price agreed upon at the time the reservation was made. The application must enforce business rules that cannot be guaranteed by field constraints alone. For example, reservations and service-bay time overlaps require checks that account for existing active records and concurrent requests. Inventory validation and corresponding inventory updates must also be coordinated so that two concurrent transactions cannot oversell the same products. These operations should use database transactions to keep related changes consistent.

6.4 Data flows

The application modules use the shared database to coordinate inventory, reservations, orders, and repairs. Core business records for each store are maintained in the same database that are used by each of the four store locations.

For rental reservations, the application reads reservation and bike information to determine availability across the requested rental period. The reservation process uses RESERVATION and RESERVATION_BIKE to associate the customer, reservation, and individual bikes. Before a reservation is confirmed, the application must recheck availability to prevent conflicts. When a retail bike is returned, the application updates its rental and bike status. If the bike requires repair, it is marked unavailable and a repair ticket is created.

For repairs, the servicing module creates a REPAIR_TICKET associated with the bike and handling store. It uses SERVICE_BAY and SERVICE_APPOINTMENT to schedule work without conflicting appointments. When parts are consumed, the application records the usage in REPAIR_PART_USAGE and deducts the corresponding quantity from INVENTORY_RECORD as one operation. When the repair is completed, the repair and bike statuses are updated so the bike can become available for rent again.

For retail orders, the application checks store inventory before accepting an order and records the order as well as order lines. Stock validation and order creation are validated to prevent orders from exceeding available inventory.

Important changes are recorded in AUDIT_LOG to support accountability and troubleshooting. The shared database allows the relevant modules to read consistent business records while preventing reservation, order, and repair history.

7. Quality and Test Plan

7.1 Quality approach

The team has decided to use quality assurance and testing throughout Scrum-based development rather than waiting until the end of the project. This approach prioritizes the risks identified in Week 3: inconsistent inventory across four stores, overlapping rental reservations, rental bikes being offered while under repair, double booking service bays, unauthorized access, and slow availability searches.

Quality assurance focuses on preventing defects through requirements of review, code reviews, consistent coding and documentation standards, continuous integration, and an agreed definition of what done is. Before a feature enters developments, the team will review its user story and acceptance criteria for completeness, consistency, and testability. Changes to the shared relational database, availability rules, and module interfaces will receive particular attention because they can affect multiple modules and store locations.

Testing by executing the system with selected inputs works well because we can compare actual results with expected results. The team will use unit, integration, system, scenario, security, performance, and useability testing. A feature is only complete when its interface works, and when business rules, authorization, error handling, and acceptance criteria are verified.

7.2 Testing in the Scrum process

Testing will take place during each sprint. Before work begins, the team will identify acceptance criteria, relevant risks, and how the feature will be tested. During development, the module owner will perform unit tests and check normal and invalid inputs. A peer reviewer will also examine important code, design, data changes, and related tests before integration. Integration testing will verify interactions between modules and the database. Examples include confirming repair updates remove a bike from rental availability, repair part usage reduces the correct store's inventory, an order affects product availability, and reservation changes are visible across the four stores. The team will also run scenario tests for rental reservations, online orders, and rental bike repairs.

Nonfunctional tests will be performed when the relevant features are available. Performance testing will use representative data from all four stores, concluding approximately 400 rental bikes and up to 100 concurrent users. Security testing will verify role based permissions, while usability testing will measure whether trained employees can complete assigned primary tasks within five minutes without assistance.

A story will not be considered done if acceptance criteria are unmet, required tests haven't been run or results were not as expected, or the change breaks existing required functions. High risk items such as reservations, inventory, repairs, sand service bays will be prioritized first for full regression and performance testing

7.3 Test ownership and timing

Stage	Assignee	Task
Before Development	Whole Team	Review requirements, acceptance criteria, risks, and test approaches.
During Development	Module Owner	Run unit tests and check normal and invalid inputs.
Before Integration	Peer Reviewer	Review code, design, data changes, and scenario tests.

During Integration	Module Owner	Test interactions between modules and the shared database.
Before Approval	Whole Team	Run acceptance and scenario tests.
Before Production	Whole Team	Review test results, unresolved issues, and database changes

7.4 Detailed high-risk tests

The team has selected five detailed tests based on the highest priorty requirements and the main risks identified in Week 3.

Test 1: Prevent overlapping rental reservations

The tester will have two customers or two customer accounts to attempt to reserve the same bike for overlapping periods, including requests submitted concurrently. The system must perform a final availability check and prevent conflicting reservations from being confirmed. The test fails if the database does not reflect the outcome or if both customers receive a confirmation message.

Test 2: Prevent rental of bikes under repair

The tester will create a repair ticket and mark a bike as under repair. They will then use a customer account to search for that bike during a period when it would otherwise be available. The bike must be excluded from rental results until the repair is completed and the bike is able to return to be rented. The test fails if the bike can be reserved while under repair or its rental availability does not match its repair status.

Test 3: Verify data consistency across all four stores

The tester will record initial inventory values, perform dummy transactions, create reservations, repairs, service appointment updates, and measure how long the changes take to become visible on the server or at other locations. At least 99% of affected updates must become visible within 10 seconds when network connectivity is available. The test fails if the measured rate falls below that 99% or outdated information is presented as current without an error and a timestamp of the last successful update.

Test 4: Prevent double booking service bays

Two employees or a tester using two employee accounts will attempt to schedule appointments in the same service bay during overlapping periods, including concurrent requests. The first appointment must remain valid, and the conflicting appointment must be rejected with a message mentioning the scheduling conflict. The test fails if both appointments are confirmed, the database does not reflect the outcome, or if a service bay is marked as available when it isn't.

Test 5: Enforce customer and employee access permissions

Customer account #1 will attempt to access Customer account #2's order, reservation, or repair information by changing a record identifier in the URL or by requesting a direct URL. An employee will

also attempt an action reserved for managers. The system must deny all unauthorized attempts without exposing protected information, while allowing each role to perform its permitted tasks. The test fails if protected data is returned, or an unauthorized action goes through.

Along with these five tests, the team will test product search, order quantities, and the full repair/service workflow. Performance testing must demonstrate that at least 95% of inventory, rental, and service bay searches are completed within three seconds under the agreed workload. Usability testing must verify that trained employees can complete assigned tasks within five minutes without assistance.

7.5 Additional traceability coverage

Beyond the high-risk tests, the team will verify that each in-scope functional requirement has corresponding test coverage. Additional cases include:

- Searching products by name or identifier and displaying correct store-level availability.
- Creating, modifying, and cancelling rental requests.
- Recording order totals and progressing order through pickup statuses.
- Restricting customers to their own information.
- Updating repair tickets and completing the inspection workflow.
- Viewing service appointments by store, date, time, and service bay.
- Confirming that unauthorized operations are rejected consistently across Web/API layer.

Nonfunctional testing will cover specified performance, consistency, availability, security, and usability targets.

7.6 Requirements-to-test traceability

The team will maintain a traceability record where each test result is linked to the corresponding Week 3 requirements and acceptance criteria. This allows the team to identify uncovered requirements and distinguish between tests that have been written and tests that have been passed.

Requirement	**Verification Approach**
FR-01: Product search and store level availability	Functional tests for search results and inventory quantities
FR-02 and FR-06: Rental availability and reservation management	Functional and concurrency tests for rental periods, changes, cancellations, and conflicting reservations.
FR-03 and FR-08: Repair tickets and customer repair tracking	Scenario and integration tests for repair status, customer access, and bike availability
FR-04 and FR-05: Online orders and store pickup	Functional and integration tests for stock validation, order lines, and pickup status changes.
FR-07: Service bay appointments	Integration and concurrency tests for overlapping appointments.

FR-09: Repair part usage	Integration tests verifying part records and corresponding inventory deductions
FR-10: Manager reservation approval	Tests verifying that availability is checked again when a manager approves a pending reservation.
NFR-01: Search performance	Performance tests against the three second target for at least 95% of requests
NFR-02: Cross store consistency	Timed tests against the 99% within the 10 seconds target
NFR-03: Availability	Monitoring and outage records evaluated against the monthly availability target
NFR-04: Access control	Authorization tests confirming that unauthorized attempts are denied
NFR-05: Usability	Observed task completion tests with trained employees

7.7 Escaped defects and remediation

An escaped defect is a problem discovered after a change reaches the production environment. Examples include conflicting rental reservations, a bike being rented while under repair, incorrect inventory, double booked service bays, or unauthorized access to customer information.

The person reporting a defect should record the function, store, reproduction steps, expected and actual results, occurrence time, relevant reservation, order, repair, bike or product identifiers, and any available error messages, logs, or screenshots. The issue will be recorded in the team's tracking system and linked to the affected requirement or acceptance criteria.

The coordinator or support contact will notify the Lead Engineer and Project Manager. The lead engineer will assess the technical impact, while the team evaluates business impact and priority. Defects that threaten customer information, corrupt data, allow conflicting reservations, permit rental of unavailable bikes, or disruption of core operations will be considered critical and escalated immediately.

For a critical defect, the team will limit further damage if possible, by either temporarily disabling affected reservations, stopping faulty inventory updates, removing bikes from availability, or preventing further use of a conflicting service bay. The lead engineer will identify the cause and propose the smallest safe fix. Except when emergency action is necessary to prevent greater harm.

The fix will be developed and reviewed in the development environment before moving through quality assurance and staging. If data has become inconsistent or corrupted, the team will identify affected records and review them before closing the issue. Whenever possible, someone other than the person who implemented the fix will verify it by reproducing the original issue, running the relevant acceptance and regression tests, checking affected database records, confirming cross store consistency, and verifying that the corrective action is recorded in the audit log.

After closure, the team will review how and why the defect escaped and determine what needs to change in our practices to prevent something like this happening again.

7.8 Testing limits and quality gates

Testing alone is not enough to prove that there are zero issues. The system may still encounter unexpected usage patterns, hardware failures, network interruptions, or data quality issues. The team will reduce these risks through risk-based testing, controlled releases, monitoring, backups, and documented recovery procedures.

The release must not proceed if critical business rules fail, authorization tests reveal unresolved access-control vulnerabilities, or transactional operations permit overselling or conflicting reservations. The release must also be reviewed with nonfunctional targets evaluated. Performance results, availability, evidence, consistency behavior, and usability results should be documented rather than assumed. If a target cannot be demonstrated, the team should record the gap.

8. Reconciliation: What the Team Found and Changed

When the team combined the weekly project components, we found several differences in workflow descriptions, interface assumptions, data relationships, and quality targets. We resolved them by treating the approved scope and functional requirement as the basis for the integrated design, then updating the relevant interface, data design, and test plan descriptions to match.

Area	Earlier project direction	Current integrated plan
Overall goal	Centralize inventory, rental sales, and repairs across four stores	Same goal while preventing inaccurate availability, overselling, double bookings, and scheduling conflicts
User roles	Customers and store staff were the main users	Three roles are defined: customers, employees, and managers with server wide access controls
Rental reservations	Support bike availability and reservations	Check availability, submit a pending request, let a manger approve it, and prevent overlapping active reservations
Repairs	Track repair work and keep bikes under repair unavailable	Add service-bay scheduling, parts deductions, repair completion checks, and customer access to their own repair status.
Online sales	Support online product sales and store pickup	Define stock checks, order details, and statuses, eligible pickup stores, and employee updates to ready for pickup and collected
Data and architecture	A centralized system	Chose modular n-tier monolithic application and an authoritative

		relational database with asynchronous workers for work that does not need to block a transaction
Boundaries	Broad business capabilities and exclusions	Explicitly exclude areas such as payroll, suppliers, shipping logistics, a dedicated mobile app, separate employee-account business entities, and unneeded analytics.
Quality goals	Quality concerns such as consistency, security, performance, and availability were identified	Add measurable targets: three-second searches for 95% of requests, cross store updates within 10 seconds for at least 99% of updates, 99.5% availability during published operating hours, role based security tests, and usability tests.

The project evolved from a broad business concept into a design with defined workflow, data relationships, measurable acceptance criteria, and explicit technical risks. The new detail should be described as refinement and reconciliation rather than changing our core purpose.

9. Deployment and Maintenance Outlook

9.1 Release approach

The system should be released incrementally rather than introducing every feature at once. The team should first establish authentication and server-side authorization, the shared relational database, and core business rules. The initial application should connect inventory search with rental reservation requests and approval, while also supporting repair status and the rule that a bike under repair cannot be offered for rentals.

Before each release, the team should deploy to a staging environment and run unit tests, integration tests, regression tests, security checks, and relevant performance tests. Release checks should verify that database storage works as expected, and inventory and reservation transactions preserve consistency, role restrictions are enforced, and the main user journeys meet their acceptance criteria. The team should also verify backups, monitoring and recovery procedures before deployment to production.

After the initial application is stable, later increments can add or expand online orders and store pickup, service bay scheduling, repair part inventory, deductions, and customer repair requests. Features should be released only when their dependencies and acceptance criteria are satisfied.

9.2 Rollback plan

If a release causes serious issues, the team should first assess the impact and restrict or disable the affected operations when possible to do safely. The release owner should review application errors, test results, database changes, and the scope of affected records before deciding whether to roll back.

If the previous application version is compatible with the current database schema, the team can redeploy that version and run smoke tests on critical workflows. Database migrations should be designed and reviewed carefully so that an application rollback does not leave the previous version unable to read or update the data. If a rollback causes corrupted data, restoring or recovering the affected data from backups and available recovery records may be necessary. Simply reverting the application will not repair incorrect database records.

After rollback, the team should confirm that inventory, reservations, orders, repair statuses, and service bay schedules are correct across all affected stores. The team should document the incident, communicate any remaining limitations, fix the root cause, and rerun relevant tests before attempting another release.

9.3 Hardest maintenance areas

The most difficult maintenance work will involve preserving consistency across services that share the same data. For example, a repair ticket must make a rental bike unavailable, a completed repair must not make a bike available prematurely, and parts used during a repair must reduce the correct store's inventory. Reservation approval must also prevent conflicting reservations, including concurrent requests. These rules cross module boundaries and require reliable transactional behavior and regression testing.

A second challenge is maintaining the shared database and its schema as the system evolves. Because the modules use one shared database, changes to entities or relationships can affect several areas of the application. The team will need version-controlled schema migrations, test data, compatibility checks, backups, and documented recovery procedures.

The third challenge will be meeting availability and consistency targets in production. The application and shared database simplify cross-store coordination, but they create central dependencies where application or database failures can affect multiple stores at once. Monitoring, backup verification, recovery drills, capacity checks, and review of unresolved incidents will be important ongoing responsibilities.

The selected architecture is manageable for the expected scale of four stores and approximately 400 rental bikes, but it depends on careful transaction design, disciplined database changes, and documented recovery practices.

10. Conclusion

The Golden Bike Shop system will provide customers, store employees, and managers with one coordinated view of rentals, repairs, service bays, online orders and pickups across all four store locations. The application will be built using a modular n-tier monolith architecture with a shared

relational database because the most important business rules depend on accurate, consistent information across multiple workflows and stores.

The team would first build authentication and server-side authorization, the shared database, inventory search, rental reservation requests and manager approval systems, and repair status handling. This first build would establish the data and business rules on which the remaining services and features depend on. It also gives the team an early opportunity to test features like cross-store consistency, access restrictions, reservation conflicts, and making sure bikes cannot be rented when they are under repair. Once this is stable, the team can expand into online orders, store pickup, service bay scheduling, and repair parts usage. Building these connected capabilities first reduces the risk of adding more features on top of inconsistent data or incomplete business rules.